

TESTE DE PERFORMANCE 5



Engenharia de Software

Projeto de Bloco: Engenharia de Softwares Escaláveis

Aluna: Rafaela Oliveira Ribeiro

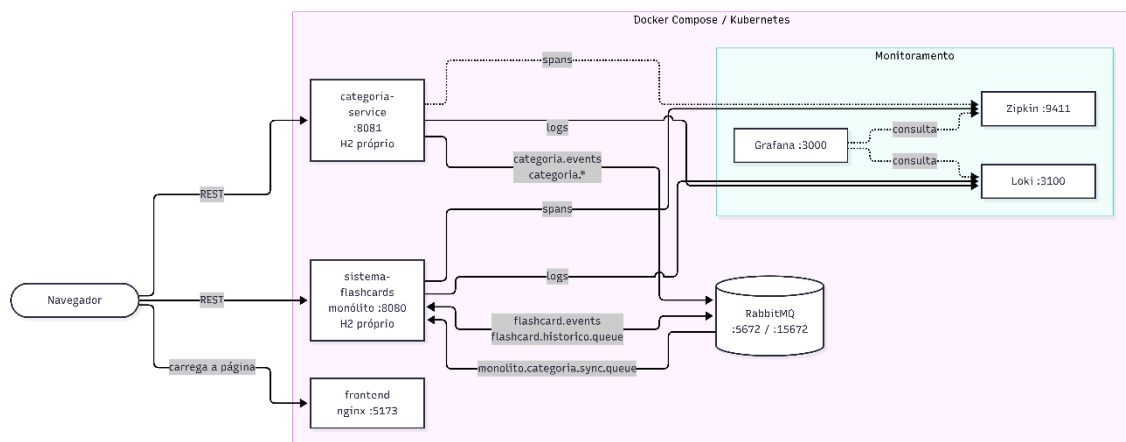
25/09/2026

Visão geral

Nesta etapa o sistema de flashcards foi preparado para operação. Os três componentes (frontend React, monólito sistema-flashcards e microsserviço categoria-service) ganharam imagens Docker e foram implantados num cluster Kubernetes local. Também foram configurados logs agregados, rastreamento de transações, um pipeline de CI/CD no GitHub Actions e testes de integração.

Arquitetura

A comunicação entre os serviços continua a mesma da Etapa 4, com eventos pelo RabbitMQ. O que entrou nesta etapa foram o Zipkin, o Loki e o Grafana, e todos os componentes passaram a rodar em containers.



Containerização com Docker

Os Dockerfiles do monólito e do categoria-service fazem o build em duas etapas. A primeira compila com Maven e a segunda, baseada em `eclipse-temurin:21-jre`, copia só o `.jar`, então a imagem final não leva Maven nem código-fonte. O frontend segue a mesma ideia, com build no Node e os arquivos estáticos servidos pelo nginx.

O `docker-compose.yml` sobe o sistema inteiro. O RabbitMQ e o monólito têm healthcheck, e o categoria-service só sobe depois do monólito estar saudável. Na Etapa 4 vimos que os eventos das categorias de exemplo se perdem se forem publicados antes de a fila do monólito existir, e essa ordem evita o problema.

Implantação no Kubernetes

O cluster usado é o Kubernetes do Docker Desktop, que enxerga as imagens geradas localmente. Os manifestos ficam em `k8s/`, numerados na ordem de aplicação (namespace, monitoramento, RabbitMQ, monólito, categoria-service e frontend), e os Services do tipo LoadBalancer publicam as portas no localhost.

Os serviços Java têm `readinessProbe` e `livenessProbe` nos endpoints do Actuator. Assim o pod só recebe tráfego quando está pronto e é reiniciado se travar. O `categoria-service` tem um `initContainer` que espera o monólito ficar pronto, pelo mesmo motivo da ordem no `compose`.

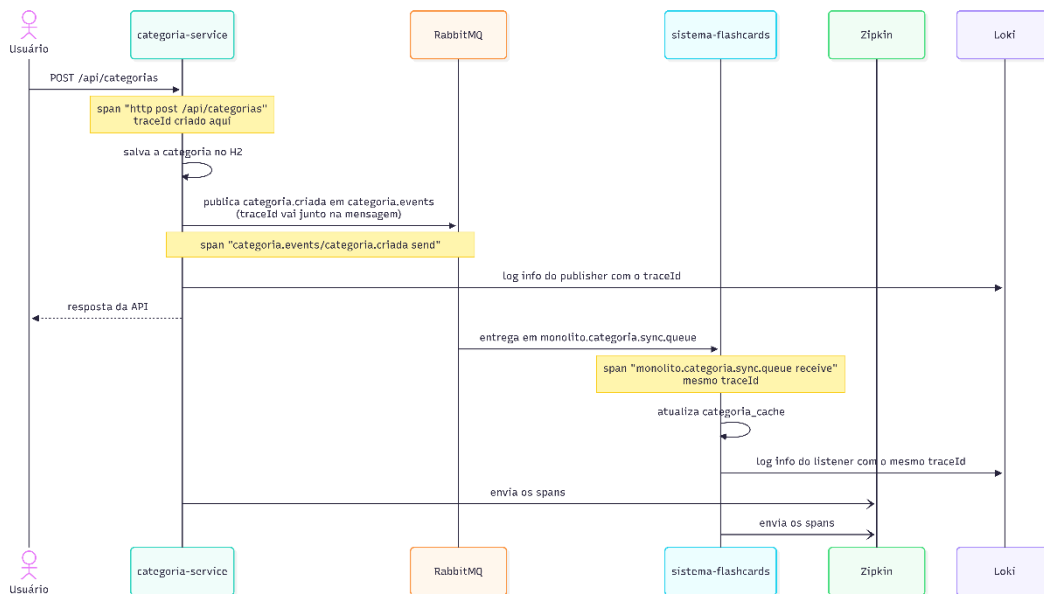
Testei a orquestração de duas formas. Ao apagar o pod do monólito, o Deployment criou outro sozinho. Ao escalar o frontend de 2 para 3 réplicas, o Service passou a distribuir as requisições entre os novos pods.

Monitoramento

Os dois serviços receberam o Actuator, o `spring-boot-starter-zipkin` (Micrometer Tracing) e o `loki-logback-appender`. A configuração principal fica no `application.properties`.

```
management.endpoints.web.exposure.include=health,info
management.tracing.sampling.probability=1.0
management.tracing.export.zipkin.endpoint=http://localhost:9411/api/v2/spans
spring.rabbitmq.template.observation-enabled=true
spring.rabbitmq.listener.simple.observation-enabled=true
loki.url=http://localhost:3100/loki/api/v1/push
```

As duas propriedades `observation-enabled` fazem o `traceId` viajar dentro da mensagem do RabbitMQ. Com isso, um `POST /api/categorias` aparece no Zipkin como um único trace que começa no `categoria-service`, passa pelo broker e termina no consumo pelo monólito. A amostragem de 100% serve para a demonstração, não para produção.



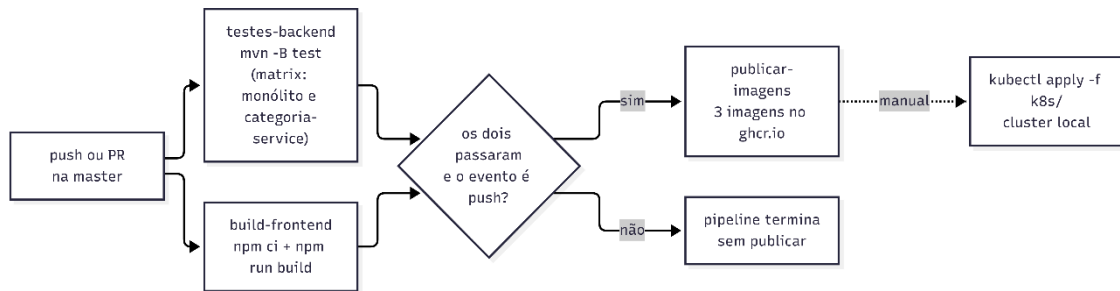
Os logs vão para o Loki quando o perfil docker está ativo, com os labels app e level e o traceId na mensagem. No Grafana dá para filtrar por serviço ou buscar um traceId e ver os logs dos dois serviços da mesma transação. O appender ficou restrito a esse perfil porque, sem o Loki no ar, ele quebrava os testes.

Configuração e versionamento

A mesma imagem roda localmente, no compose e no Kubernetes. O que muda entre os ambientes são as variáveis `SPRING_PROFILES_ACTIVE`, `SPRING_RABBITMQ_HOST`, `MANAGEMENT_TRACING_EXPORT_ZIPKIN_ENDPOINT` e `LOKI_URL`, que o Spring Boot lê no lugar das propriedades. Sem elas, os valores padrão apontam para localhost.

O código e os arquivos de infraestrutura (Dockerfiles, compose, manifestos e workflow) estão versionados juntos no GitHub, com um commit por etapa ou alteração.

CI/CD com GitHub Actions



O workflow `.github/workflows/ci-cd.yml` roda a cada push e pull request na master. Na parte de CI, o job `testes-backend` roda `mvn -B test` nos dois serviços Java e o `build-frontend` confirma que o React compila. Na parte de CD, o job `publicar-imagens` roda só em push e só se os anteriores passarem, e publica as três imagens no GitHub Container Registry.

O deploy no Kubernetes continua manual, com `kubectl apply`, porque o cluster é local e o GitHub não tem acesso a ele.

Testes

São 30 testes no monólito e 17 no `categoria-service`, todos com H2 em memória, sem depender de RabbitMQ, Zipkin ou Loki. Os unitários testam serviços, repositórios e consumidores de mensagens isoladamente, com Mockito e `@DataJpaTest`.

Nesta etapa entraram os testes de integração `FlashcardIntegrationTest` e `CategoriaIntegrationTest`. Eles sobem a aplicação com `@SpringBootTest` e `MockMvc` e testam o ciclo completo pela API (criar, buscar, atualizar, deletar e 404). No monólito, um deles verifica também que o flashcard devolve o nome da categoria a partir da tabela de cache.

A integração entre os serviços foi verificada com o sistema rodando no compose e no Kubernetes. Uma categoria criada no `categoria-service` aparece com o nome correto num flashcard do monólito, e o Zipkin mostra o trace passando pelos dois.

Evidências

← → ↻ http://localhost:15672/#/queues

RabbitMQ

RabbitMQ 3.13.7 Erlang 26.2.5.16

Overview

Connections

Channels

Exchanges

Queues and Streams

Admin

Queues

▼ All queues (3)

Pagination

Page 1 of 1 - Filter: ☐ Regex ?

Overview					Messages			Message rates			+/-
Virtual host	Name	Type	Features	State	Ready	Unacked	Total	incoming	deliver / get	ack	
/	flashcard.historico.queue	classic	D DLX DLK	running	0	0	0	0.00/s	0.00/s	0.00/s	
/	flashcard.historico.queue.dlq	classic	D	running	0	0	0				
/	monolito.categoria.sync.queue	classic	D	running	0	0	0	0.00/s	0.00/s	0.00/s	

Zipkin

Find a trace

Dependencies

Search by trace ID

EN

⚙

serviceName categoria-service x spanName http post /api/categorias x +

RUN QUERY

⚙

Results

EXPAND ALL

COLLAPSE ALL

Service filters

Root	Start Time	Spans	Duration	
categoria-service: http post /api/categorias	5 minutes ago (09/25 16:10:25:370)	3	40.144ms	SHOW
categoria-service (2) rabbitmq (2) sistema-flashcards (1)				
categoria-service: http post /api/categorias	5 minutes ago (09/25 16:09:51:106)	3	19.023ms	SHOW
categoria-service (2) rabbitmq (2) sistema-flashcards (1)				
categoria-service: http post /api/categorias	2 minutes ago (09/25 16:12:48:815)	3	16.486ms	SHOW
categoria-service (2) rabbitmq (2) sistema-flashcards (1)				
categoria-service: http post /api/categorias	4 minutes ago (09/25 16:11:29:023)	3	14.145ms	SHOW
categoria-service (2) rabbitmq (2) sistema-flashcards (1)				

Zipkin

Find a trace

Dependencies

Search by trace ID

EN

⚙

Start Time 09/24/2026 16:08:16

End Time 09/25/2026 16:15:16

RUN QUERY

Filter

```
graph LR; categoria-service((categoria-service)) --> sistema-flashcards((sistema-flashcards)); sistema-flashcards --> rabbitmq((rabbitmq));
```

```
Problems 5 Output Debug Console Terminal Ports

Pool: DataSourceConnectionProvider
Minimum pool size: undefined/unknown
Maximum pool size: undefined/unknown
2026-09-25T16:27:39.708-03:00 INFO 17664 --- [main] [
'hibernate.transaction.jta.platform' to enable JTA platform integration)
2026-09-25T16:27:39.716-03:00 INFO 17664 --- [main] [
ersistence unit 'default'
2026-09-25T16:27:39.844-03:00 WARN 17664 --- [main] [
ult. Therefore, database queries may be performed during view rendering. Explicitly configure
2026-09-25T16:27:40.066-03:00 INFO 17664 --- [main] [
tuator'
2026-09-25T16:27:40.080-03:00 INFO 17664 --- [main] [
2026-09-25T16:27:40.101-03:00 INFO 17664 --- [main] [
ctory#64231d5d:0/SimpleConnection@6c8d6a2f [delegate=amqp://guest@127.0.0.1:5672/, localPort=
2026-09-25T16:27:40.132-03:00 INFO 17664 --- [main] [
in 1.192 seconds (process running for 14.723)
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 1.237 s -- in com.rafa
[INFO]
[INFO] Results:
[INFO]
[INFO] Tests run: 30, Failures: 0, Errors: 0, Skipped: 0
[INFO]
[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
[INFO] Total time: 21.146 s
[INFO] Finished at: 2026-09-25T16:27:43-03:00
[INFO] -----

C:\Users\User\Documents\INFNET\ENGENHARIAS ESCALÁVEIS\PROJETO DE BLOCO\sistema-flashcards>
```

```
categoria-service
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.873 s -- in com.rafaelaribeiro.categori
[INFO] Running com.rafaelaribeiro.categoria_service.CategoriaServiceTest
[INFO] Tests run: 8, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.324 s -- in com.rafaelaribeiro.categori
[INFO]
[INFO] Results:
[INFO]
[INFO] Tests run: 17, Failures: 0, Errors: 0, Skipped: 0
[INFO]
[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
[INFO] Total time: 16.044 s
[INFO] Finished at: 2026-09-25T16:29:25-03:00
[INFO] -----

C:\Users\User\Documents\INFNET\ENGENHARIAS ESCALÁVEIS\PROJETO DE BLOCO\sistema-flashcards\categoria-service>
```

Observação: outras evidências foram gravadas durante o processo de gravação do vídeo da Entrega do Projeto de Bloco.

Link do Repositório Github

<https://github.com/RafaelaRibe1ro/flashcards-sistema>